GETTING STARTED WITH VERSION CONTROL: INTRO TO GIT AND GITHUB

DATA CULTURE SOCIETY

Support for data-led and applied digital research across the arts, humanities and social sciences.

THE UNIVERSITY of EDINBURGH
Centre for Data, Culture & Society

DATA CULTURE SOCIETY

Course Designers

Ponrawee Prasertsom

PhD student in Linguistics, PPLS

Joy Lan

PhD student in Digital Education,
Moray House School of Education

Original materials developed by Aislinn Keogh

THE UNIVERSITY of EDINBURGH
Centre for Data, Culture & Society

The University of Manchester

Today's Instructor

Phil Reed

Research Community and Training Manager,
The University of Manchester

Various helpers

Volunteers from DH & RSE Summer School

Plan for Today

09:10 Welcome & introduction

09:30 *Live Demo & troubleshooting*: Create your first repo & your first commit

10:00 *Live Demo & troubleshooting*: Create a new branch and merge it

10:30 Break

11:00 *Live Demo & troubleshooting*: Fork a repo & create a pull request

11:30 *Live Demo & troubleshooting*: Resolving a merge conflict

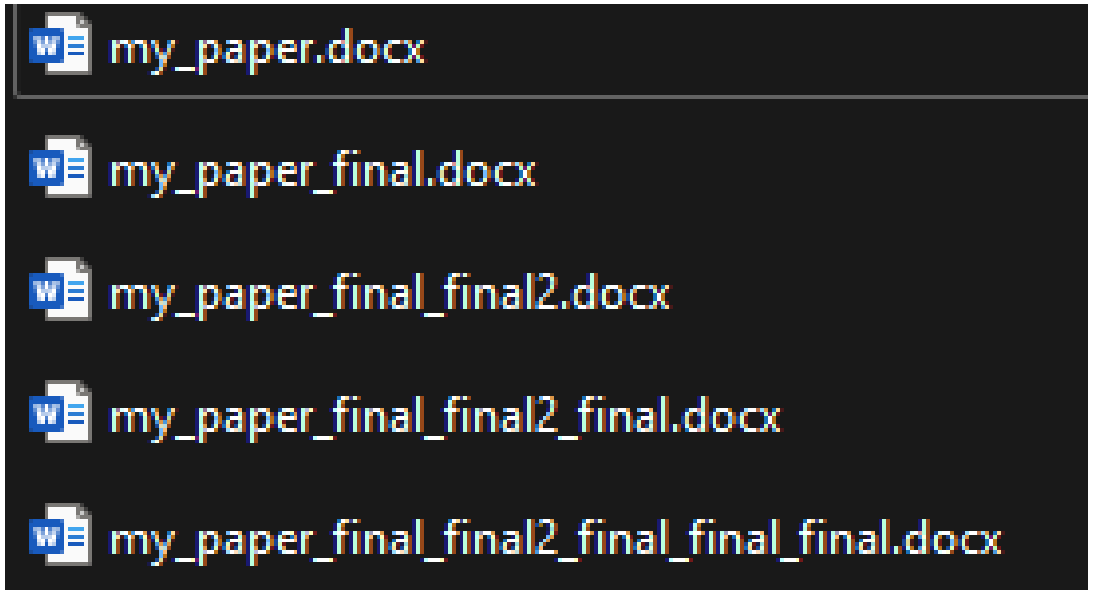
12:00 Generative AI and GitHub

12:10 Wrap up & additional resources

12:30 End of session / Lunch

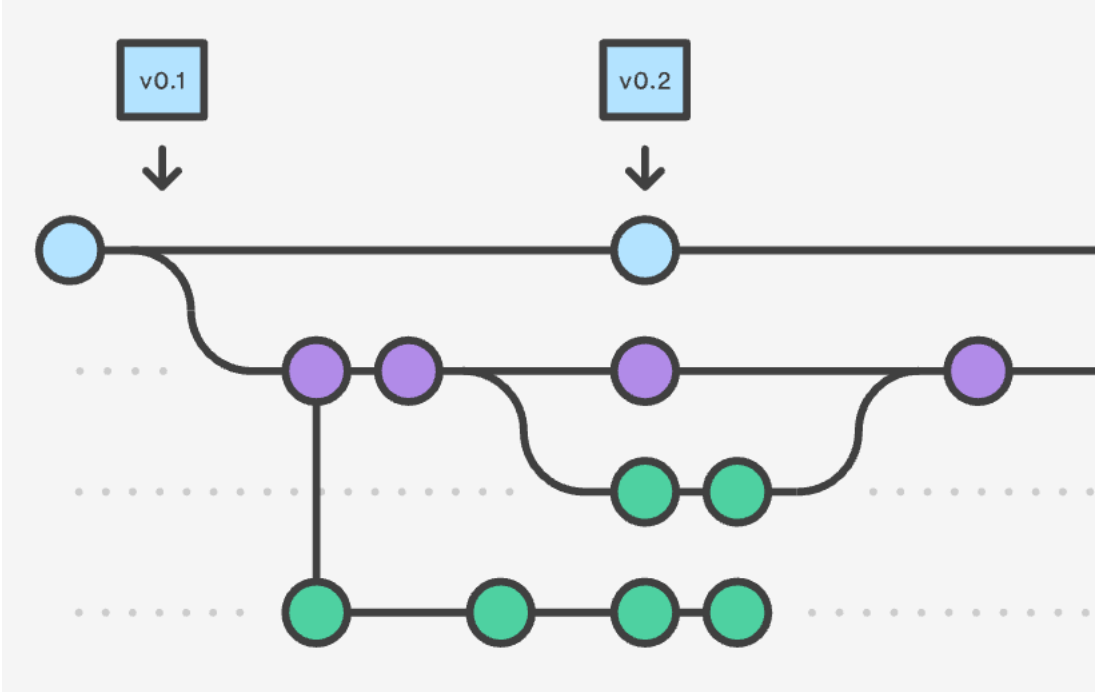
Version Control

- **Version control** helps you track changes to your files.
- You probably already have done (poor) version control before!



Version Control

- **Version control is the practice of tracking and managing *changes* to files** (traditionally, software code).
- Version control tracks every individual change by each contributor and helps prevent ***concurrent*** work from conflicting.
- It is a standard ***workflow*** in the tech industry.



git

Git is a version control system installed and maintained on your local system

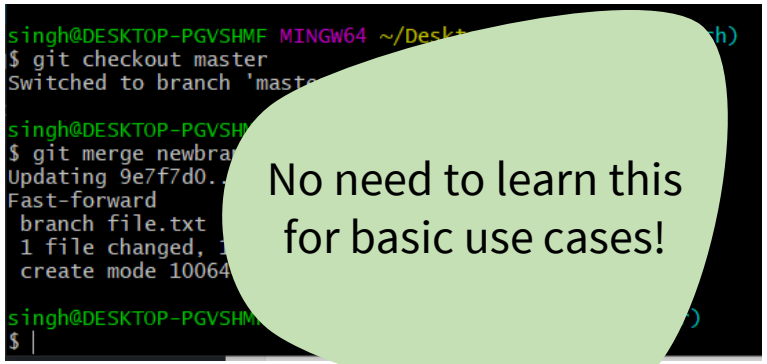
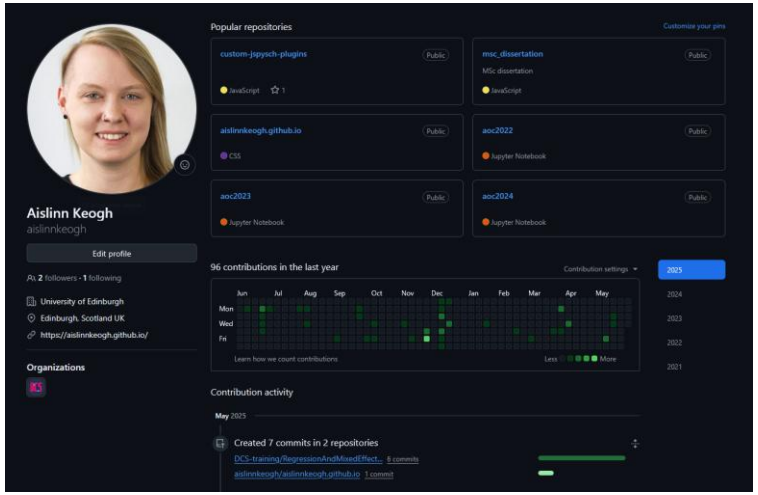


Image from https://funkyvast.weebly.com/what-is-git-bash-terminal.html

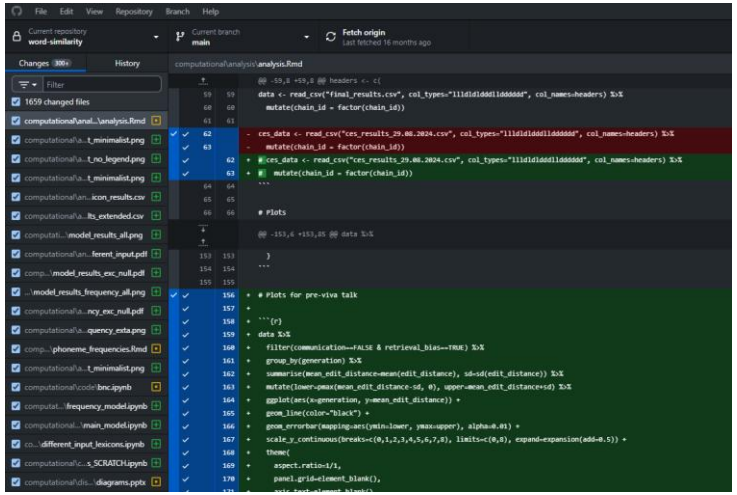
GitHub

GitHub is a hosting service for Git repositories.
It is exclusively cloud-based



GitHub Desktop

A desktop application for working with git and GitHub on your computer



Terminology

repository/repo

commit

clone

branch

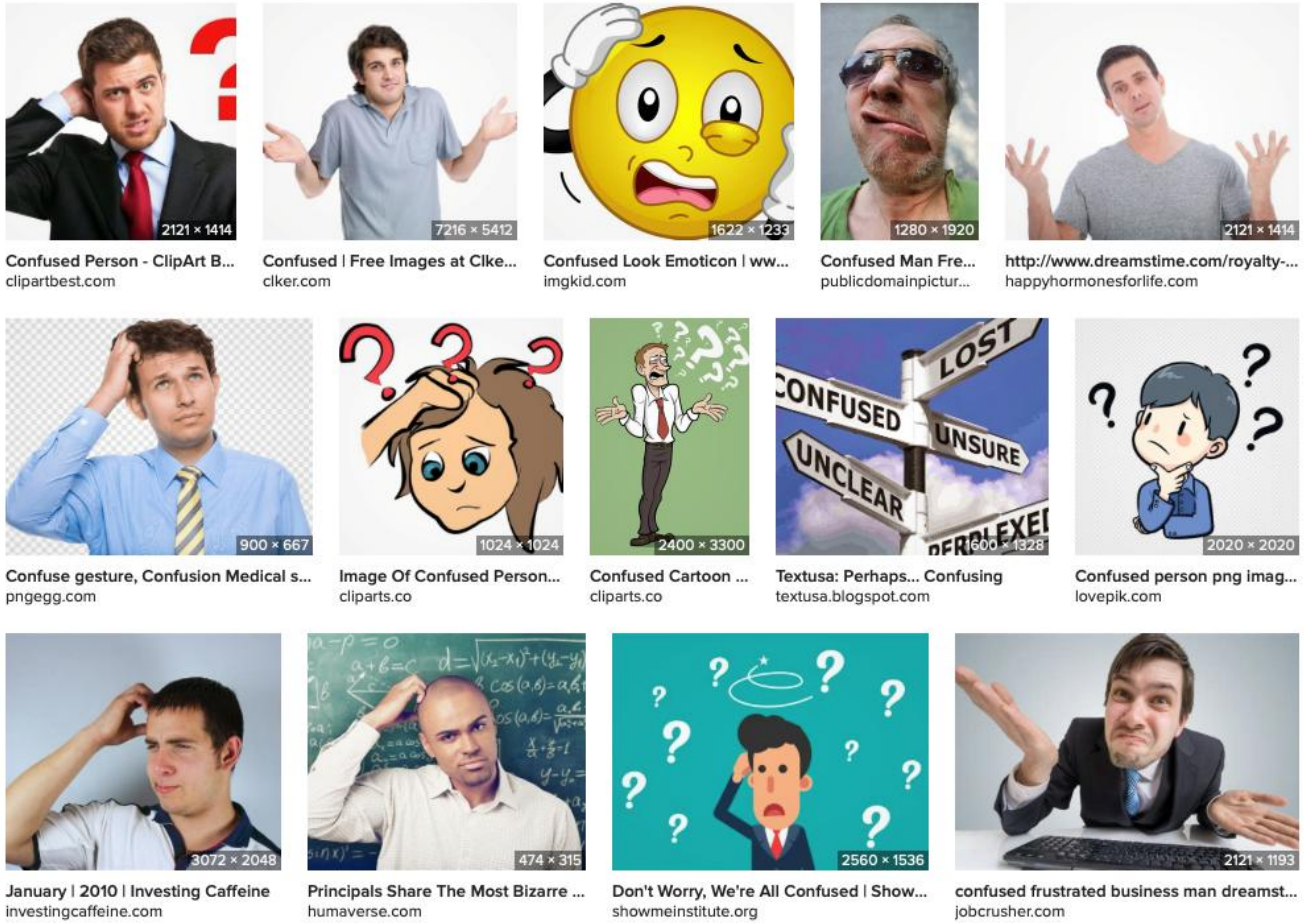
merge

fetch

pull

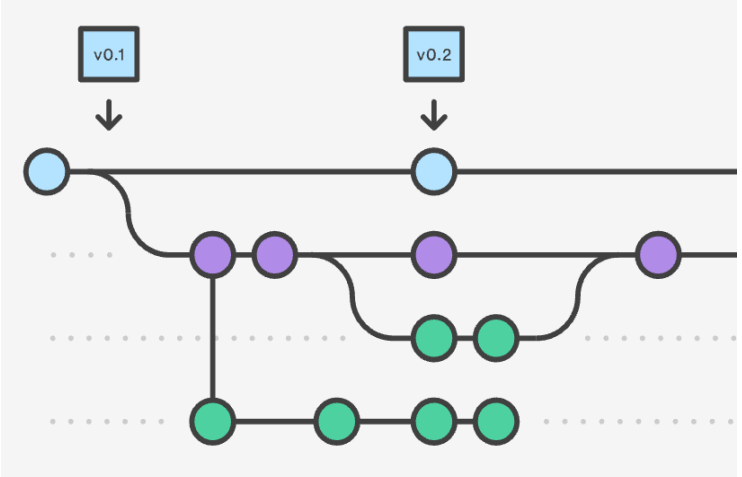
push

fork



Terminology

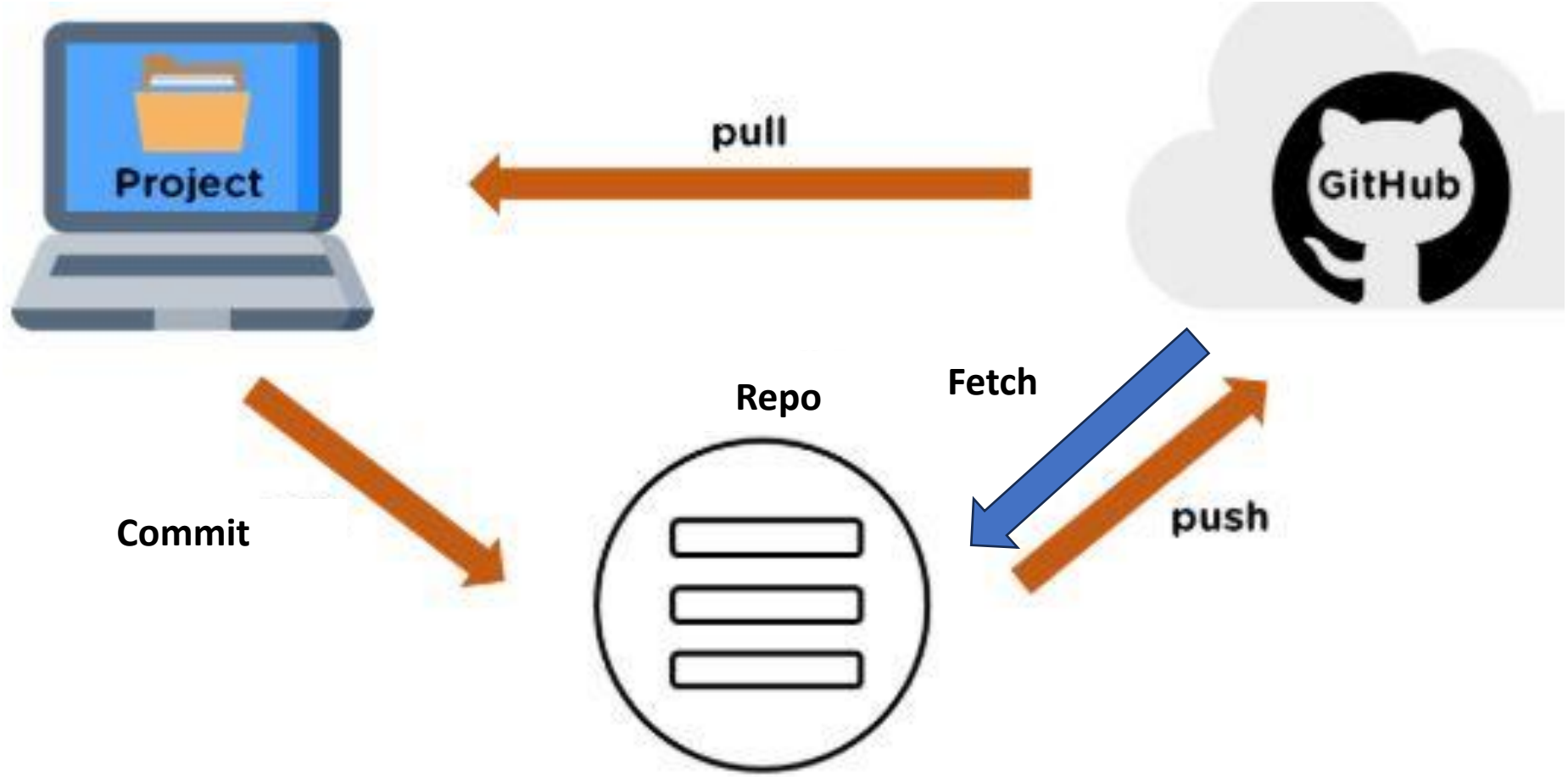
repository/ repo → a “folder” for your code for one project
It's a place where you can store your code, your files, and each file's revision history. Repositories can have multiple collaborators and can be either public or private.



clone → a copy of a repo (e.g. on your computer)

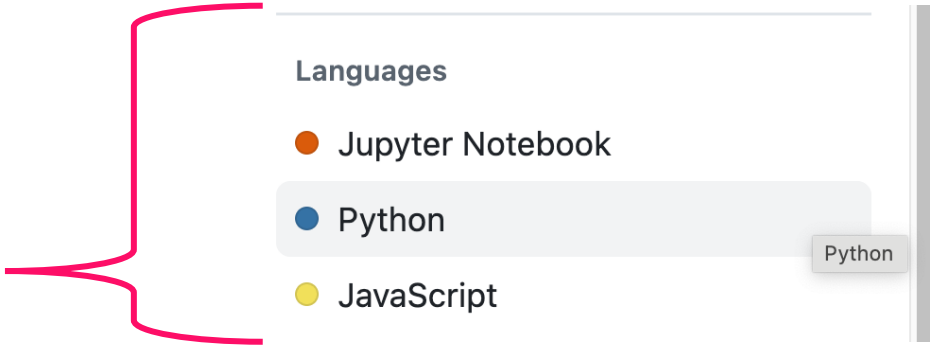
branch → a parallel version of a repo (repos can have multiple branches)

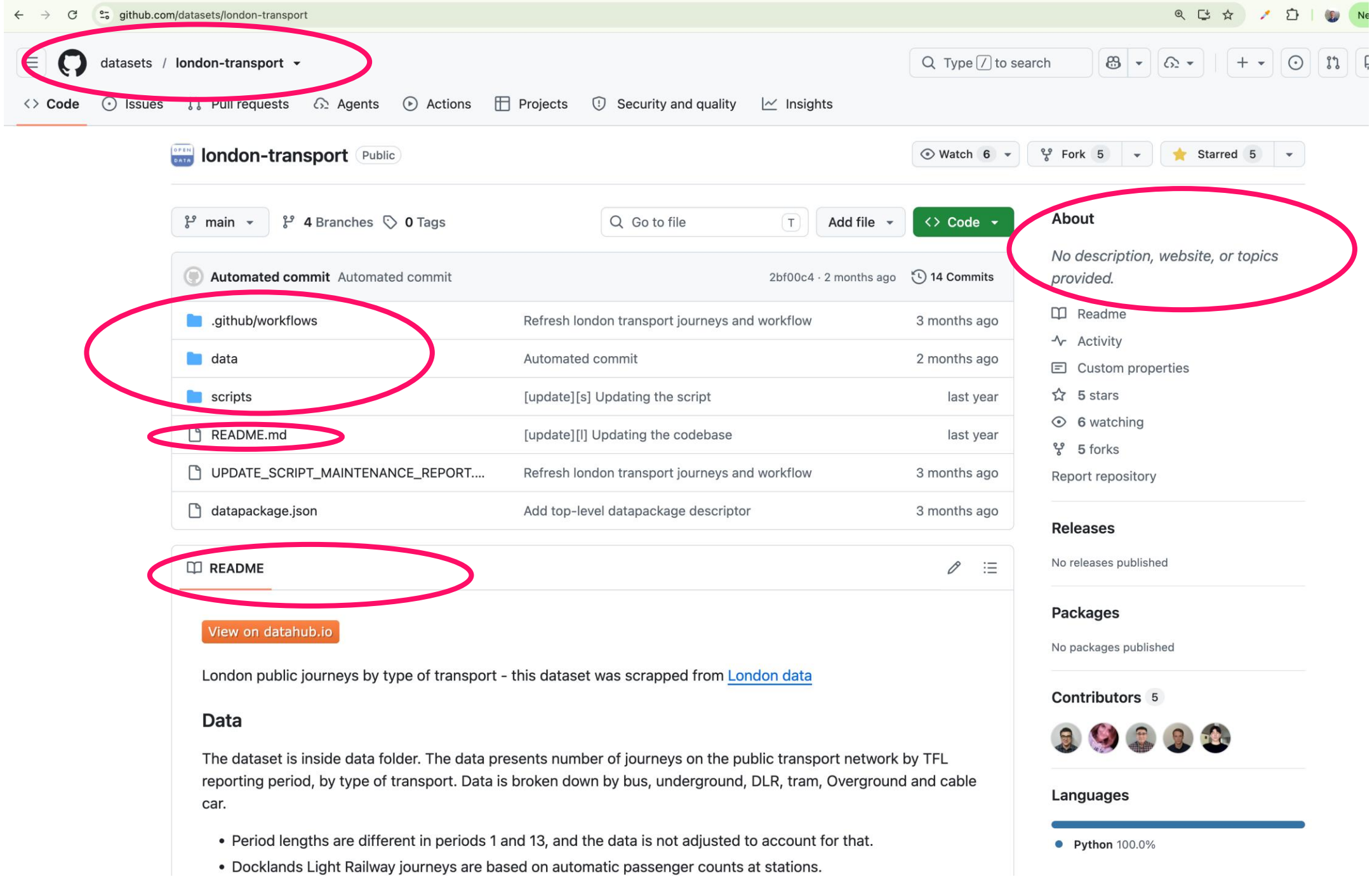
Terminology



Viewing an existing repo

1. Go to www.github.com
2. Search for “London transport”
3. Add a Language filter for “Python”
4. Select “ datasets/london-transport ” (updated 1 May or later)
   https://github.com/datasets/london-transport
5. Describe what sort of files and folders it contains.




github.com/datasets/london-transport
datasets / london-transport
Type / to search
Code
Issues
Pull requests
Agents
Actions
Projects
Security and quality
Insights
london-transport
Public
Watch 6
Fork 5
Starred 5
main
4 Branches
0 Tags
Go to file
T
Add file
Code
Automated commit
Automated commit
2bf00c4 · 2 months ago
14 Commits
.github/workflows
Refresh london transport journeys and workflow
3 months ago
data
Automated commit
2 months ago
scripts
[update][s] Updating the script
last year
README.md
[update][l] Updating the codebase
last year
UPDATE_SCRIPT_MAINTENANCE_REPORT....
Refresh london transport journeys and workflow
3 months ago
datapackage.json
Add top-level datapackage descriptor
3 months ago
README
View on datahub.io
London public journeys by type of transport - this dataset was scrapped from London data
Data
The dataset is inside data folder. The data presents number of journeys on the public transport network by TFL reporting period, by type of transport. Data is broken down by bus, underground, DLR, tram, Overground and cable car.
Period lengths are different in periods 1 and 13, and the data is not adjusted to account for that.
Docklands Light Railway journeys are based on automatic passenger counts at stations.
About
No description, website, or topics provided.
Readme
Activity
Custom properties
5 stars
6 watching
5 forks
Report repository
Releases
No releases published
Packages
No packages published
Contributors 5
Languages
Python 100.0%

Cloning a repo with GitHub Desktop

1. Install GitHub Desktop from https://desktop.github.com if you haven't yet
2. Open GitHub Desktop, and log in with your account.
3. Locate the GitHub repository and click 

then "Clone: Open with GitHub Desktop".
4. Choose the location (local path) of your repo on your machine and proceed.
5. Describe what sort of files and folders it contains.

Creating your first repo

1. Go to www.github.com
2. Sign in / sign up
3. Click the **+** in the top right corner and select “New repository”
4. Choose a repository name, check “Add a README file”, and leave all other fields.
5. Click “Create repository”

Cloning your repo with GitHub Desktop

1. Install GitHub Desktop from https://desktop.github.com if you haven't yet
2. Open GitHub Desktop, and log in with your account.
3. Locate your GitHub repository and click 

then "Clone: Open with GitHub Desktop".
4. Choose the location (local path) of your repo on your machine and proceed.

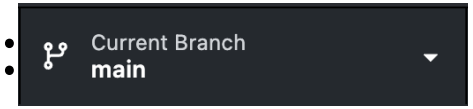
Making a commit

1. Open the README.md file in a text editor of your choice
   If you're unsure: on Windows use **Notepad**; on Mac use **TextEdit**
   - If you can't find the README.md file in your repo folder, you can create a README.md file using the editor.
2. Make some changes in your README file and save them
3. In GitHub Desktop, describe your commit and click commit when ready
4. Push to GitHub

Terminology

repository/repo	→	a “folder” for your code for one project
clone	→	a copy of a repo (e.g. on your computer)
commit	→	saving your changes in the project
push	→	uploading your changes (your commits) to the server
fetch	→	downloading the latest changes from the remote repo
pull	→	integrating the latest changes into your clone of the repo
branch	→	a parallel version of a repo (repos can have multiple branches)
merge	→	combine the changes from one branch into another
fork		

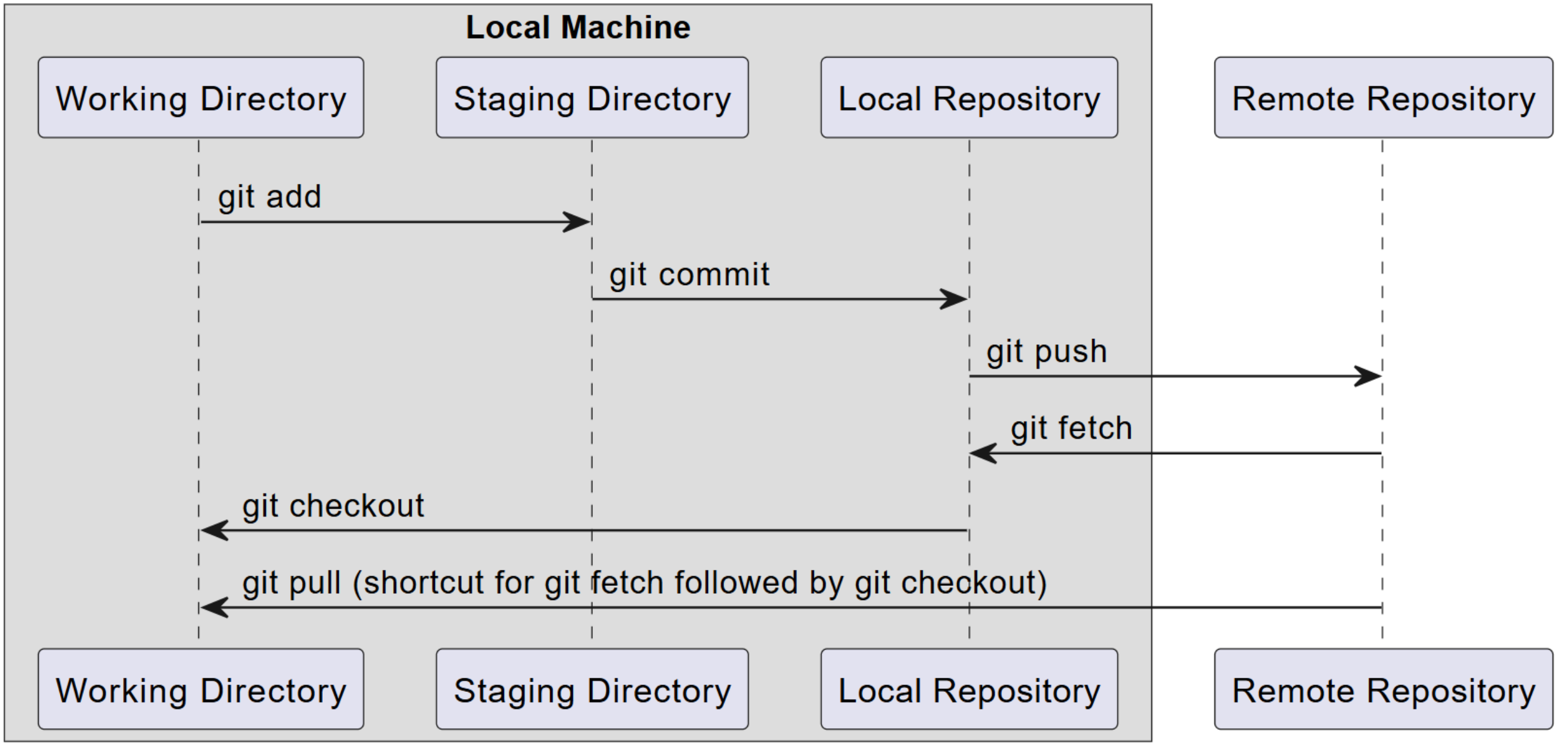
Creating a new branch and merging it

1. In GitHub Desktop, open the branch menu: 

2. Select ‘new branch’ and enter a name of your choice. Confirm that this branch is now displayed under ‘Current Branch’.
3. Publish the new branch.
4. Make some changes in your README file. Save, commit, push.
5. Go to Branch -> Merge into the current branch and select your new branch. Create a merge commit, then push.

Break

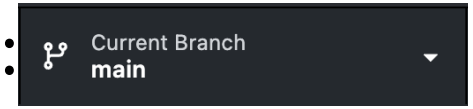

GETTING STARTED WITH
VERSION CONTROL: INTRO
INTRO TO GIT AND GITHUB



Terminology

repository/repo	→	a “folder” for your code for one project
clone	→	a copy of a repo (e.g. on your computer)
commit	→	saving your changes in the project
push	→	uploading your changes (your commits) to the server
fetch	→	downloading the latest changes from the remote repo
pull	→	integrating the latest changes into your clone of the repo
branch	→	a parallel version of a repo (repos can have multiple branches)
merge	→	combine the changes from one branch into another
fork	→	a personal copy of someone else’s repo

Creating a new branch and merging it

1. In GitHub Desktop, open the branch menu: 

2. Select ‘new branch’ and enter a name of your choice. Confirm that this branch is now displayed under ‘Current Branch’.
3. Publish the new branch.
4. Make some changes in your README file. Save, commit, push.
5. Go to Branch -> Merge into the current branch and select your new branch. Create a merge commit, then push.

Live Demo:
Forking a repo, making a commit, and making a pull request

1. Find a partner and get the link to their GitHub repo. Fork it on GitHub, then add your fork to GitHub Desktop.
2. Make a change to their README, commit, and push.
3. On GitHub, view your fork and open a pull request.
4. Review and approve each other's pull requests.

Live Demo:
Resolving a merge conflict

1. Choose your or your partner's repo.
2. In your fork of your partner's repo, make changes to README, commit, and push to your own fork.
3. Your partner also makes changes to the README, commit, and push to their own repo.
4. Make a pull request for your partner's repo. This will create a merge conflict. Resolve as demonstrated.

Generative AI and GitHub

There are many ways that people use **GitHub Copilot**, including:

1. Get **code suggestions** as you type
2. **Chat** with Copilot to get help (browser/command line)
3. Generate **descriptions of changes** in a pull request
4. Create pull requests for you to **review** (and vice versa)

Alternatives: Claude Code, Cursor and others

Generative AI and GitHub

Why use it?	How to be careful	Why not use it?
• Productivity boost • Integration • Learning tool • Built-in security	• Validate every line • Turn off auto complete • Data privacy • Check local rules	• Subtle errors • Project too complex • Costs • Cloud dependency

Additional Resources

- GitHub online tutorials
  - Introduction (tutorial repeating some of what we did today): https://github.com/skills/introduction-to-github
  - Reviewing pull requests: https://github.com/skills/review-pull-requests
  - Resolving merge conflicts: https://github.com/skills/resolve-merge-conflicts
  - List of all skills tutorials: https://github.com/skills
- GitHub student pack: https://education.github.com/pack